

ringio

Kernel-Bypass Lock-Free Asynchronous I/O Engine for High-IOPS NVMe Storage

Standalone Systems Research & Engineering Blueprint | Domain: Storage Microarchitecture & Low-Latency Systems

[EXECUTIVE ARCHITECTURAL VISION]

ringio is a header-only C++20 zero-allocation asynchronous storage engine built to eliminate operating system kernel overhead in modern high-throughput NVMe SSD workloads. By bridging user-space lock-free MPMC ring buffers directly with Linux io_uring kernel submission and completion queues operating in SQPOL mode, ringio delivers true kernel-bypass file I/O performance and predictable sub-microsecond latency.

1. Project Mission & Individual Research Scope

ringio is an independent, low-level C++ systems engineering research initiative designed to demonstrate that a single engineer can build a production-grade, zero-overhead storage engine that outperforms standard operating system I/O abstractions.

The primary objective is to engineer a clean, header-only library that unlocks maximum hardware performance from enterprise NVMe storage media while producing peer-reviewable, empirically sound microarchitectural benchmarks.

2. The Hardware Bottleneck & Theoretical Foundations

With the advent of modern PCIe Gen 4/5 enterprise NVMe SSDs delivering millions of Input/Output Operations Per Second (IOPS) and single-digit microsecond hardware access times, the primary bottleneck in storage software has fundamentally shifted from physical flash media latency to **operating system kernel overhead**.

Traditional synchronous file I/O abstractions (POSIX `read`/`write`, `pread`/`pwrite`) and legacy asynchronous APIs (`libaio`) introduce four major microarchitectural penalties under high-concurrency workloads:

- **System Call Overhead & Mode Switching:** Every file operation forces execution switches between Ring 3 (user-space) and Ring 0 (kernel-space) via `syscall`/`sysret` instructions, burning 100–300 CPU cycles per invocation.
- **Thread Context Switching & Scheduler Overhead:** Blocking I/O operations cause the OS scheduler to deschedule waiting threads and execute costly register state saves, destroying CPU L1/L2 cache locality.

- **Dynamic Allocation & Memory Copying:** Standard file interfaces copy data between user-space buffers and kernel page cache structures, consuming DRAM memory bandwidth and introducing allocation stalls.
- **Page Table Translations & TLB Evictions:** The kernel must resolve virtual-to-physical address mappings on every request, pinning user pages and triggering Translation Lookaside Buffer (TLB) misses under high request rates.

3. System Architecture & Core Methodology

ringio bypasses kernel storage overhead through a pure user-space C++20 abstraction layer that communicates directly with NVMe hardware via Linux `io_uring` mechanisms:

3.1 Lock-Free User-Space Ingestion Ring

Application worker threads enqueue read/write requests into a 128-byte dual cache-line aligned lock-free MPMC queue, ensuring zero lock contention during I/O command construction.

3.2 Kernel Submission Queue Polling (SQPOLL)

ringio configures `io_uring` with `IORING_SETUP_SQPOLL` mode. A dedicated kernel thread continuously polls submission queue entries in shared memory, allowing user-space threads to issue I/O commands directly via memory writes with zero system calls.

3.3 Pre-Registered Fixed Memory Buffers

Using `io_uring_register_buffers`, ringio pre-allocates page-locked memory pools permanently mapped into DMA space. This completely eliminates runtime page pinning, memory copying, and address translation penalties during active I/O execution.

3.4 Fixed File Descriptor Registration

Using `io_uring_register_files`, file descriptors are registered at initialization, avoiding internal kernel file table reference counting and permission checks on individual operations.

4. Core Goals & Target Metrics

The goal of ringio is to demonstrate that a lightweight, custom C++20 kernel-bypass storage engine achieves superior IOPS scaling, sub-microsecond latency, and lower CPU cycle costs compared to standard Linux storage APIs on enterprise NVMe SSDs:

Evaluation Metric	POSIX <code>pread</code> / <code>libaio</code> Baseline	ringio Target Milestone
4KB Random Read Throughput	Syscall-bound (~150K–300K IOPS)	Saturates hardware NVMe queue depth (> 600K IOPS)
Tail Latency (p99 & p99.9)	Spikes to microsecond scale (>	Predictable sub-microsecond to low-

	50–100 μ s)	microsecond tail latency
System Call Count (per 1M ops)	1,000,000 system calls (1 call per POSIX op)	Near-zero system calls under steady-state SQPOLL mode
CPU Instruction Efficiency	High CPU cycle burn on context switches & TLB lookups	Minimal CPU cycle cost per completed I/O request